1er Informe del Trabajo Final
PROYECTO NAVE ASISTIDA
Programación C - 2026





Profesor:*Guccione Leonel*

Integrantes:

- *Weinbaum, Martín*
- *Gueli, Juan Pablo*
- *Mazzucco, Lisandro*
- *Bounzerio, Gonzalo*

Breve Introducción

Con motivo del Trabajo Práctico Final de la materia Programación C, de la Universidad Nacional de Mar del Plata se nos presentó la consigna de realizar un proyecto con el lenguaje Java sobre una Nave espacial con un Asistente de navegación y comandos.

El trabajo fue realizado en dos IDEs de desarrollo, NetBeans y IntelliJ IDEA, usando un repositorio de GitHub para trabajar en remoto cada integrante desde su PC. Además utilizamos la tecnología de Apache Maven para compilar y mantener una estructura de proyecto común entre IDEs.

Cabe aclarar que fue utilizado el uso de distintas Inteligencias Artificiales para guiar el camino del proyecto. Ver uso uso-ia.md en el repositorio.

Link al repositorio GitHub:

https://github.com/tinchoweinbaum/tpg-progra3

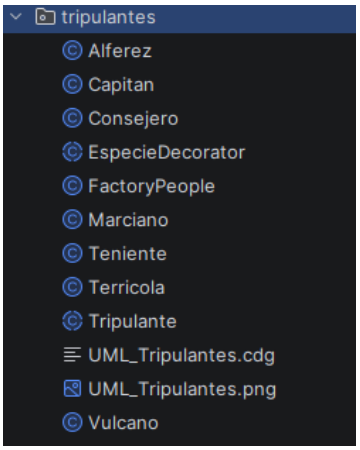
Desarrollo

Iniciamos el proyecto desde la clase con menos responsabilidades, la tripulación. Uno de los requisitos para formar esta parte del proyecto era usar el Patrón Decorator visto en clase para calcular los sueldos de cada tripulante, según sea su cargo u orígen. Partimos de una clase abstracta de la cual se extienden los cargos y más adelante el Decorador con sus respectivas *clases Origen.* Más adelante fusionamos la creación en un mismo método aplicando el Patrón Factory, para abstraer al usuario de la creación y que no tenga que saber a qué clase llamar para crear un tripulante de cierto tipo, solo asignarle al método creador, la combinación deseada.

Siempre teniendo en cuenta los patrones SOLID para la elaboración del TP, en este caso puntual podemos decir que estamos cumpliendo la letra “O”, la “L”, y la “D”, es decir:

- “O” - Principio Abierto/Cerrado
    - Un código puede ser extendible pero no modificable; podríamos agregar tipos de especies nuevas o más cargos, sin tener que corregir todo el código
- “L” - Principio de sustitución de Liskov
    - Los objetos pueden ser reemplazados por instancias de sus subtipos; en los métodos que incluyen a los tripulantes trabajamos con la clase base *Tripulante* , no con algún objeto de cargo y/o especie específico, sino que la compilación del programa nos brinda las herramientas que, junto con nuestro código, nos permiten a acceder a los datos particulares de un tripulante específico
- “D” - Principio de Inversión de Dependencias
    - Se depende de abstracciones, no de implementaciones; como mencionamos anteriormente, el usuario no sabe, y no debe saber a qué instancia de clase u objeto invocar para la creación de un personaje, solo se deja llevar por el andar del programa y le solicita que le devuelva un objeto Tripulante del cargo y especie solicitado

Captura de las clases y archivos del paquete Tripulantes:



Captura del Patrón Factory aplicado a la creación de los personajes:

```java
public static Tripulante creaPersonaje(String cargo,String origen,String nombre,int antiguedad) throws ErrorCreacionPersonajeException{
    Tripulante aux = null;
    switch (cargo.toUpperCase()) {
        case "CAPITAN":
            aux = new Capitan(nombre,antiguedad);
            break;
        case "ALFEREZ":
            aux = new Alferez(nombre,antiguedad);
            break;
        case "CONSEJERO":
            aux = new Consejero(nombre,antiguedad);
            break;
        case "TENIENTE":
            aux = new Teniente(nombre,antiguedad);
            break;
        default:
            throw new CargoInvalidoException("El cargo '" + cargo + "' no existe");
    }

    switch (origen.toUpperCase()) {
        case "TERRICOLA":
            return new Terricola(aux);
        case "MARCIANO":
            return new Marciano(aux);
        case "VULCANO":
            return new Vulcano(aux);
        default:
            throw new OrigenInvalidoException("El origen '" + origen + "' no existe");
    }
}
```

Más adelante seguimos con las creaciones de cada tipo de Nave, en este trabajo contamos con las establecidas por la consigna, naves de tipo Carguero, de Combate, o Exploradoras. Con un comportamiento análogo a la creación de personajes, se usó el Patrón Factory, aunque en este caso era requisito de la consigna.

Cada uno de las naves al crearse comienza con distintos valores de energía y combustible, establecidos por la rúbrica de la cátedra. A continuación se muestran los constructores necesarios para crear cada nave, que son herencias de la clase abstracta *Nave*, y descienden en clases concretas para cada tipo.

```java
public abstract class Nave{  19 usages  martin linux +3
    public static final float MAX_COMBUSTIBLE = 100;  4 usages
    public static final float MAX_ENERGIA = 100;  4 usages
    public static final float MAX_DESGASTE = 100;  3 usages

    protected float combustible, energia, desgaste = 0;  7 usages
    protected ArrayList<Tripulante> tripulantes = new ArrayList<>();
    protected Tripulante capitan = null;

    public Nave(){  12 usages  GonzaloBounzerio
        super();
    }
```

```java
public class NaveExploradora extends Nave{  2 usages  GonzaloBounzerio +3

    /**
     * Contructor de la nave explorador
     * <b>Post:</b> Combustible = 60 y energia == 80
     */
    public NaveExploradora(){  3 usages  GonzaloBounzerio +1
        super();
        this.setCombustible(60f);
        this.setEnergia(80f);
    }
}
```

Cada clase concreta , que hereda de la clase *Nave* conoce los métodos de esta última, y usa los necesarios para *settear* sus valores de recursos iniciales. Seguimos respetando principios SOLID, como la “O” , “L” y “D”, igual que en el apartado anterior con los personajes.

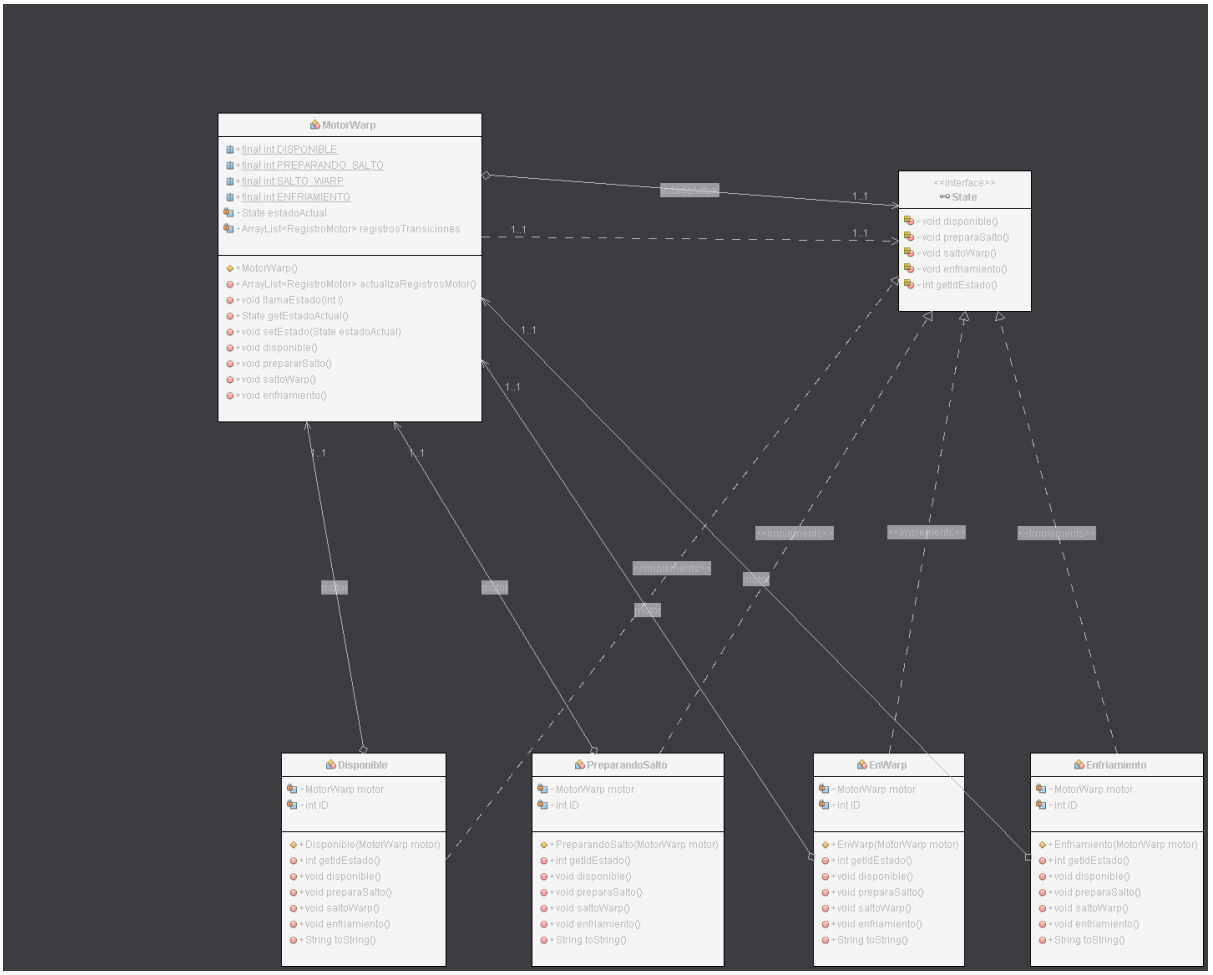
Para la construcción del *Motor Warp* se utilizó el Patrón State para pasar entre los distintos estados del motor, estos son:

- Disponible
- Preparación del salto
- Salto Warp
- Enfriamiento

Como no contamos con la información y los detalles para causar problemas en la navegación, cada estado pasa automáticamente al otro, esperando un tiempo de 500 milisegundos como para ofrecer un poquito más de realismo, y que no sean todas secuencias instantáneas.

Existen transiciones válidas en el orden detallado anteriormente, y finalmente, el estado de *Enfriamiento* vuelve al estado *Disponible*. En caso de querer ir hacia un estado con una transición ilegal, el programa arroja una excepción de error y no cambiará de estado.

A continuación el diagrama UML de este paquete para observar las relaciones principales entre estados, motor, y la interfaz state:



Todos estos cambios de estado del motor, la actualización de recursos de la nave, y el manejo de las misiones, las cuales comentaremos más adelante, deben quedar registradas en algún lado como para tener un historial de acciones de cada nave, en este caso, de la que controla el jugador. Para eso, creamos la clase *Bitácora* , la cual contiene todos los tipos de mensajes posibles que queramos guardar para luego acceder a ellos y hacer un recorrido por los escenarios que involucran a la nave. Creamos distintos tipos de registros:

- Bitácoras de motor
- Bitácoras de misiones
- Bitácoras de errores
- Bitácoras de actualización de recursos

Cada una de estas bitácoras contiene información del evento, guardando una fecha para luego obtenerlas por orden al consultar la bitácora. La función que se encarga de agregarla es la siguiente, usando conceptos de *Colecciones*.

```
public void agregarRegistro(RegistroBitacora registro) {  15 usages  Lisandro-M +1
    assert registro != null : "El registro a agregar no puede ser nulo";
    assert invariant();

    int pos = Collections.binarySearch(this.registros, registro);
    int indiceInsercion;

    //Si la busqueda binaria no encuentra un elemento devuelve su presunta posicion - 1
    if (pos < 0){ // Si no lo encuentra, lo que idealmente pasaria siempre, calculamos el indice en base al retorno
        indiceInsercion = -pos-1;
    } else { // Si por algun motivo la fecha es exacta, insertamos uno al lado del otro, sin importar el orden
        indiceInsercion = pos;
    }
    this.registros.add(indiceInsercion, registro);

    assert invariant();
}
```

Se puede apreciar también el uso de asertos, invariantes en los métodos, estos, juntos con los contratos nos han hecho más fácil poder invocar métodos de clases a otras cuando estábamos programando, ya que nos daban una ayuda de que debe recibir un método y que tipo de objeto devolverá, o qué operación realizará.

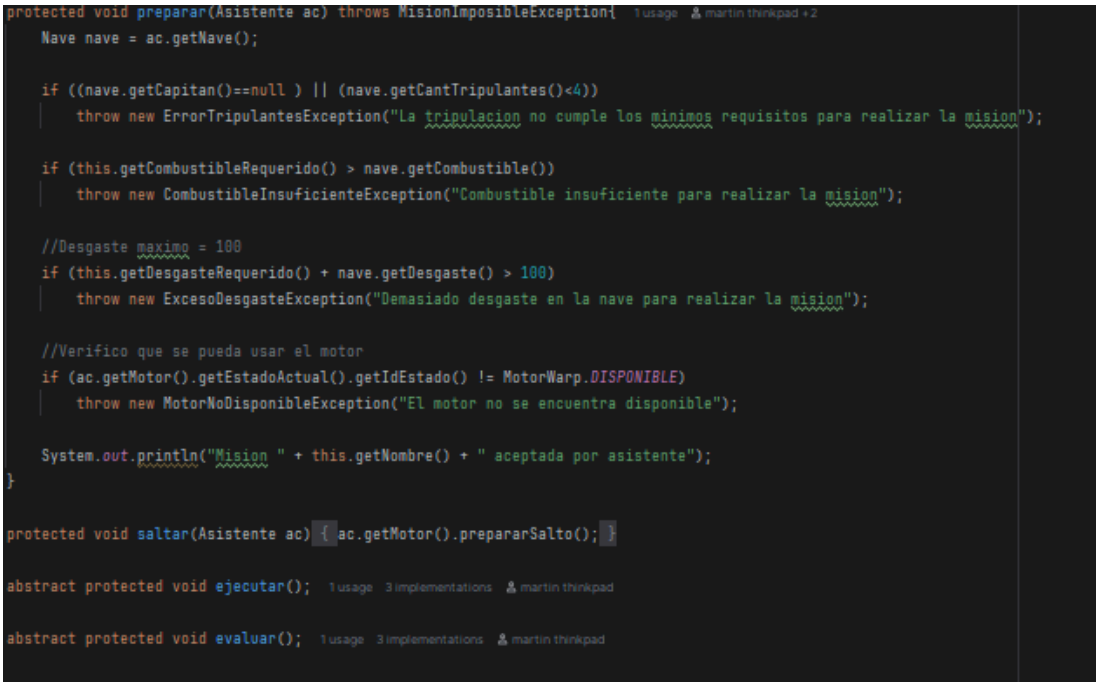
Un ejemplo de un contrato es el que escribimos para el método que crea los registros de errores. Como se menciona en el uso-ia.md del repositorio, algunos contratos fueron mejorados con llm de navegador, o en otros casos, nos indicaron falencias y puntos a retocar.

```
/**
 * Constructor que crea un registro de bitácora asociado a una excepción
 * <b>pre:</b> fechaRegistro != null && descripcionRegistro != null && descripcion!= "" y error != null
 * <b>post:</b> getFechaRegistro() == fechaRegistro, getDescripcionRegistro() == descripcionRegistro y getError() == error
 *
 * @param fechaRegistro la fecha en la que se guarda la informacion
 * @param descripcionRegistro la informacion que guarda el registro al crearse
 * @param error la excepción capturada
 */
```

En esta primera entrega el manejo de las misiones se resumieron en la aparición en carteles en la consola ya que no tenemos las herramientas prácticas para una interfaz con la cual interactúa el usuario y el programa.

Era obligatorio el uso del Template Method para el manejo, con métodos abstractos que sean implementados por cada misión según lo requiera, y otros métodos concretos que son comunes a todas.

A continuación mostraremos los métodos que fueron hechos abstractos para luego especificarlos por misión, y los que resultan comunes:



```
protected void preparar(Asistente ac) throws MisionImposibleException{  1 usage  martin thinkpad +2
    Nave nave = ac.getNave();

    if ((nave.getCapitan()==null ) || (nave.getCantTripulantes()<4))
        throw new ErrorTripulantesException("La tripulacion no cumple los minimos requisitos para realizar la mision");

    if (this.getCombustibleRequerido() > nave.getCombustible())
        throw new CombustibleInsuficienteException("Combustible insuficiente para realizar la mision");

    //Desgaste maximo = 100
    if (this.getDesgasteRequerido() + nave.getDesgaste() > 100)
        throw new ExcesoDesgasteException("Demasiado desgaste en la nave para realizar la mision");

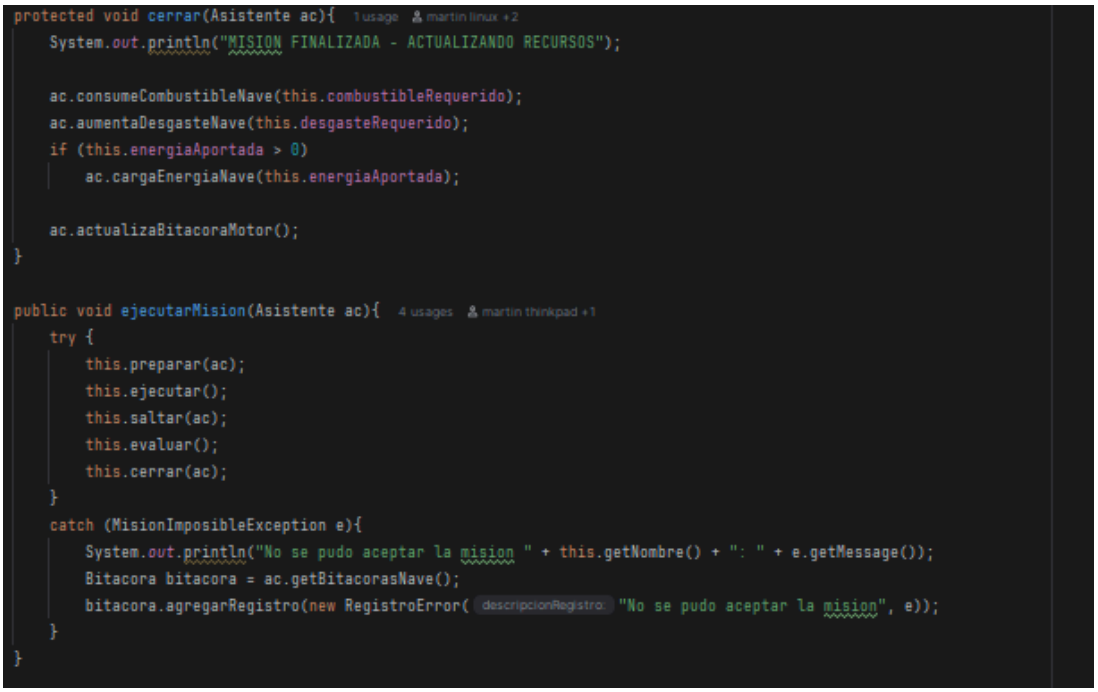
    //Verifico que se pueda usar el motor
    if (ac.getMotor().getEstadoActual().getIdEstado() != MotorWarp.DISPONIBLE)
        throw new MotorNoDisponibleException("El motor no se encuentra disponible");

    System.out.println("Mision " + this.getNombre() + " aceptada por asistente");
}

protected void saltar(Asistente ac) { ac.getMotor().prepararSalto(); }

abstract protected void ejecutar();  1 usage  3 implementations  martin thinkpad

abstract protected void evaluar();  1 usage  3 implementations  martin thinkpad
```



```
protected void cerrar(Asistente ac){  1 usage  martin linux +2
    System.out.println("MISION FINALIZADA - ACTUALIZANDO RECURSOS");

    ac.consumeCombustibleNave(this.combustibleRequerido);
    ac.aumentaDesgasteNave(this.desgasteRequerido);
    if (this.energiaAportada > 0)
        ac.cargaEnergiaNave(this.energiaAportada);

    ac.actualizaBitacoraMotor();
}

public void ejecutarMision(Asistente ac){  4 usages  martin thinkpad +1
    try {
        this.preparar(ac);
        this.ejecutar();
        this.saltar(ac);
        this.evaluar();
        this.cerrar(ac);
    }
    catch (MisionImposibleException e){
        System.out.println("No se pudo aceptar la mision " + this.getNombre() + ": " + e.getMessage());
        Bitacora bitacora = ac.getBitacorasNave();
        bitacora.agregarRegistro(new RegistroError( descripcionRegistro: "No se pudo aceptar la mision", e));
    }
}
```

```
public class Mision2 extends Mision{  2 usages  GonzaloBounzerio

    public Mision2(float combustibleRequerido, float energiaAportada, float desgasteRequerido){  3 usages  GonzaloBounzerio
        super( nombre: "M-02",  descripcion: "Recoleccion", combustibleRequerido, energiaAportada, desgasteRequerido);
    }

    public void ejecutar() { System.out.println("LLEGANDO AL PUNTO SIMULADO PARA OBTENER DATOS Y EXTRAER LA MUESTRA"); }

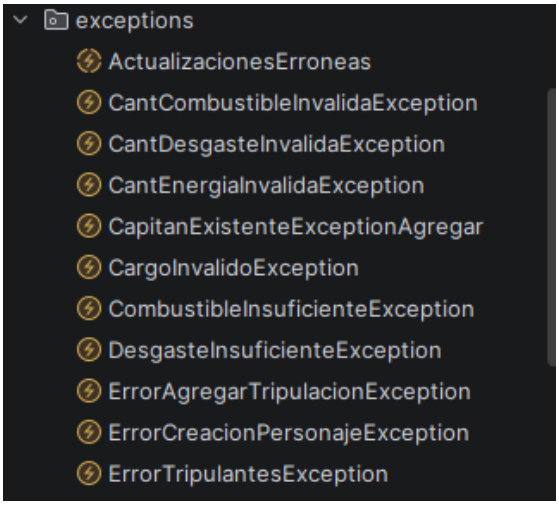
    public void evaluar() { System.out.println("EL ELEMENTO QUEDO REGISTRADO COMO OBTENIDO EXITOSAMENTE"); }
}
```

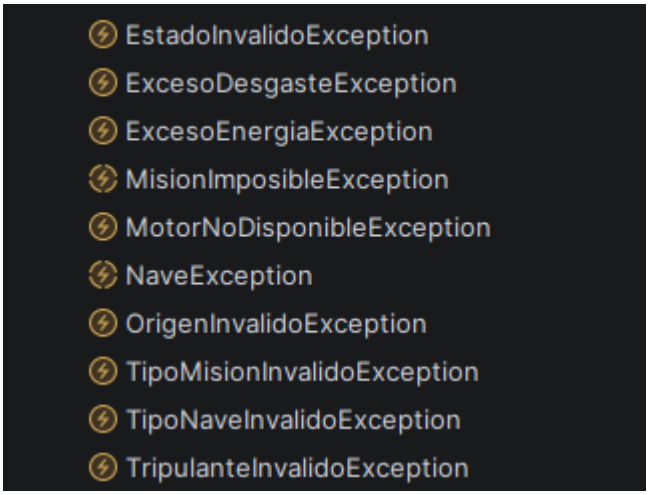
Como se puede observar, los métodos *preparar(), saltar() y cerrar()* son comunes a cada misión, en cambio, cada una imprime un mensaje distinto al invocar *ejecutar() y evaluar()*. Además existe un método concreto que se llama *ejecutarMision*, el cual accede en tiempo de ejecución(enlace tardío) al método sobrescrito de cada misión, o en su defecto al de la clase padre, *Misión.*

Este último método, recibe como parámetro al asistente que revisará los recursos de la nave y determinará si está en condiciones de poder llevar a cabo la misión según los requerimientos de la misma. Estos requerimientos se dan cuando se crean las misiones, usando nuevamente el Patrón Factory.

El asistente es la parte central del proyecto, no encapsula todas las funcionalidades de este, pero es el que se encarga de relacionar todas las partes que lo integran. Además podrá enlazarse al *Sistema* para recibir misiones, o efectuar cambios sobre su nave.

Contamos con una carpeta con todas las excepciones usadas en el proyecto, las cuales contienen un mensaje que se almacena en la bitácora para poder saber qué fue lo que falló y por qué lo hizo. El usuario nunca interactúa con ellas, pero debe saber cuándo está ocurriendo algo indebido, o contrario a lo que desea.





Conclusiones y opiniones

En el proyecto inicialmente nos repartimos en partes iguales la creación de cada parte de la nave para luego, cada día por medio, ponernos de acuerdo e implementar ideas en común tanto del diseño de la nave como algoritmos que resuelvan dicha problemática en Java. Llegando al final de esta primera entrega fue cuando más nos juntamos los cuatro integrantes del grupo y mediante llamada fuimos arreglando errores tanto conceptuales como de compilación.

Profundizamos en algunas funciones las cuales nos ayudaron a resolver partes de la bitácora, por ejemplo las colecciones, especialmente en cómo agregar registros de forma ordenada por fecha usando la búsqueda binaria. También nos resultó útil el modificar el método toString y encadenarlo con el de las superclases para que cada registro de la bitácora muestre su propia información correspondiente sin dejar de mostrar cosas básicas como la fecha o la descripción.

Y además la implementación y uso de contratos nos fue muy práctico, ya que al trabajar con otros integrantes teníamos que dejar en claro cómo funcionaba cada método con sus respectivas condiciones las cuales tenía que cumplir para que funcione correctamente.

Por último, creemos que lo más enriquecedor del trabajo práctico fue el tener que comunicarnos entre nosotros, no solo para tratar de resolver problemas del proyecto en conjunto, si no también para adaptar los métodos de nuestras respectivas partes iniciales a los requerimientos del trabajo en conjunto.